

KOSIS VDB 매칭 정확도 개선

표매칭 3단계 — 통계표 검색(VDB) 품질 문제 인수인계 문서 · 2026-08-16

이 문서는 KOSIS 표 28만7천여 개를 벡터DB(Chroma)로 검색해서 기사 claim에 맞는 통계표를 찾는 3단계 매칭 과정에서 발견된 정확도 문제들을 정리한 것입니다. 지금까지 실제로 겪은 문제(죽은 표 오매칭, 검색 자체가 정답을 못 찾는 문제)를 실측 사례와 함께 남겨두고, 추가로 필요한 개선 작업 7가지를 정리했습니다. 이 작업은 RAM이 넉넉한 환경(권장 16GB 이상)에서 진행해야 안전합니다 — 이유는 3장에서 자세히 설명합니다.

1. 배경 — 지금까지 뭐가 되어 있나

3단계(통계표 매핑)는 세 가지 방법을 합쳐서 claim에 맞는 KOSIS 표를 찾습니다: ① 64개 수동 큐레이션 카탈로그에서 키워드 매칭, ② 같은 64개 카탈로그에서 임베딩 유사도 매칭, ③ KOSIS 표 28만7천여 개 전체를 크롤링해서 만든 벡터DB(VDB, Chroma)에서 임베딩 유사도 매칭. 이 문서는 ③ VDB 쪽에 관한 것입니다.

이미 끝난 작업 (다시 안 해도 됨):

- 크롤링 — agent/kosis/crawl_table_catalog.py, KOSIS 전체 표 목록(표ID·이름·기관) 28만7천여 건 수집 완료
- 임베딩 — notebooks/vdb_embedding_colab.ipynb, 표 이름을 multilingual-e5-large로 벡터화 완료 (data/vdb_embeddings.npy, data/vdb_metadata.jsonl)
- Chroma 적재 — agent/kosis/build_vdb_index.py, 위 임베딩을 Chroma 컬렉션(kosis_vdb_tables)에 적재 완료

즉 "검색 대상 데이터베이스를 만드는 것"까지는 끝나 있고, 지금부터는 "이 검색이 실제로 정확한가"를 개선하는 단계입니다.

2. 왜 지금 이 작업이 필요한가 — 실측 사례

"지난해 1인 취업 가구는 510만 가구로 전년 대비 42만 6000가구 증가했다"라는 실제 기사 문장으로 재현한 사례입니다.

2-1. 죽은 표 오매칭

이 claim의 진짜 정답 표는 DT_1ES4I001S("1인 취업가구 현황", 2015~2025년 계속 갱신 중, KOSIS API로 실측 확인)입니다. 그런데 VDB 검색 상위 후보는:

- DT_1ES4I007 — 이름은 비슷한(" 시도별 1인 취업가구 비중") 표인데, 실제로는 2016년에 수록이 끊긴 죽은 표(KOSIS API로 확인: 2024년 조회 시 "데이터가 존재하지 않습니다" 응답)
- 이름이 "취업활동 1인가구"로 거의 동일한, 서로 다른 지자체가 낸 지역별 통계표 여러 개
임베딩 유사도만으로는 진짜 정답과 이런 죽은 표·지역 변형표를 구분하지 못합니다.

2-2. VDB 검색 자체가 정답을 못 찾는 문제 (더 심각함)

진짜 정답 표(DT_1ES4I001S)를 28만7천 개 전체와 정확히(브루트포스) 코사인 유사도 비교하면 13위(유사도 0.8439)입니다. 그런데 Chroma의 HNSW 근사검색은 top_k=50까지 넓혀도, 심지어 탐색 강도(hnsw:search_ef)를 500→5000까지 올려도 이 표를 단 한 번도 찾지 못했습니다(실측 재현 확인). ef_search를 극단적으로 올려도 안 된다는 건 그래프 "구축" 자체가 부실하다는 뜻이라 — 질의 시점엔 이미 만들어진 그래프를 탐색만 하므로, 그래프에 애초에 좋은 경로가 없으면 아무리 오래 탐색해도 소용이 없습니다.

3. 인프라 제약 — 작업 전에 꼭 알아야 할 것

위 recall 문제를 고치려면 Chroma의 HNSW 그래프를 더 촘촘한 파라미터(M, construction_ef)로 다시 만들어야 하는데, 이 과정에서 중요한 사실을 실측으로 확인했습니다:

Chroma가 28만7천 개 벡터(1024차원)를 올리면 약 4.8~4.9GB를 씁니다 — HNSW 파라미터를 뭘로 바꿔도(M=48/24, construction_ef=400/기본값 등 4가지 조합 다 테스트) 이 수치가 거의 안 바뀌었습니다. 즉 이걸 파라미터 튜닝 문제가 아니라, 이 정도 데이터량을 Chroma에 올리는 것 자체의 기본 비용입니다.

구성 요소	실측 메모리
Chroma(VDB, 28만7천 개 벡터)	약 4.8 ~ 4.9 GB
로컬 임베딩 모델(multilingual-e5-large)	약 1.3 GB
리랭커(bge-reranker-v2-m3, 참고용 추정)	약 1.5 ~ 2.5 GB

개인 노트북(예: RAM 8GB급, OS가 실제로 인식하는 건 약 7.4GB) 환경에서는 Chroma(4.8GB) + 평소 개발환경(IDE·브라우저 등, 실측 약 4~6GB)만으로 이미 여유가 거의 없어서, 임베딩 모델을 같이 로딩하면 실제로 세그멘테이션 폴트(강제 종료)가 재현됩니다. 그래서:

- 이 작업(특히 Chroma 재구축·recall 개선 실험)은 개인 노트북이 아니라 RAM 16GB 이상의 환경에서 진행하는 걸 강력히 권장합니다.
- Chroma는 원래 서버-클라이언트 구조입니다(chroma run --path ... --port ...로 띄우고 HttpClient(host=..., port=...)로 접속). 어디서 띄우든 agent/kosis/query_vdb.py의 CHROMA_HOST만 그 서버 주소로 바꾸면 나머지 코드는 그대로 씁니다 — 로컬이어야 할 구조적 이유가 없습니다.

4. 진행해야 할 작업 (8가지)

0번은 이번에 실측으로 원인까지 확인해둔 것이고(접근 방식은 검증됨, 넉넉한 메모리 환경에서 마저 진행하면 됨), 1~7번은 매칭 품질 관련 추가 문제로 논의된 항목들입니다.

작업 0 — 죽은 표 자동 배제 + VDB 검색 recall 개선

문제: 위 2장 실측 사례 참고 — 이름이 비슷한 죽은 표/지역 변형표가 섞여 있고, VDB 검색 자체도 진짜 정답을 놓치는 경우가 있음.

해야 할 일:

- 후보 표가 claim이 말하는 연도 데이터를 실제로 갖고 있는지 KOSIS API로 확인하는 로직 추가 — Param/statisticsParameterData.do를 objL1=ALL, itmId=ALL, prdSe=Y로 호출, 응답이 err=30("데이터가 존재하지 않습니다")이면 그 연도 없음으로 확정. 그 외 에러(형식 문제 등)는 판별 불가로 두고 후보를 그대로 통과시킬 것 — 확신 없는 걸 잘못 걸러내면 안 됨(옛날 기사가 옛날 표를 정당하게 가리키는 경우도 있음).
- 확인해본 후보가 전부 확실히 죽은 걸로 나오면, 조용히 아무 표나 쓰지 말고 명시적으로 "판단불가" 처리할 것 — 사실검증 시스템에서는 틀린 근거로 확신하는 것보다 모른다고 말하는 게 안전함.
- Chroma HNSW recall 결함 해결 — 넉넉한 메모리 환경에서 M/construction_ef를 충분히 올려 재구축(M=48, construction_ef=400 근처에서 시작해 recall 테스트하며 조정 권장). 위 "1인 취업가구" 사례를 회귀 테스트로 삼을 것 — 재구축 후 정답 표가 top-15 안에 들어오는지 확인.
- (참고) claim.period가 "1955~1960년"처럼 범위로 오면 연도 하나로 못 뽑아서 이 필터가 아예 안 걸리는 갭이 있음 — agent/orchestrator/slot_filler.py의 normalize_time_expressions()에 범위를 끝 연도로 접는 로직 추가가 필요.

작업 1 — 동명이표 / 유사명 오매칭

문제: "실업률"이라는 이름의 표가 KOSIS에 여러 개입니다. 전체 실업률, 청년실업률, 계절조정 실업률, 고용보조지표... 임베딩은 이름/설명이 비슷하면 다 가깝게 잡아버려서 top-k에 다 섞여 나옵니다. 기사 문장에 "청년"이라는 단어가 없으면 엉뚱한 표로 매핑될 위험이 큼니다.

방향: 표 설명에 대상 인구집단, 조사방법 등 구분 키워드를 명시적으로 metadata에 박아두고 리랭킹 단계에서 가중치를 줘야 함.

작업 2 — 지표 정의 / 단위 불일치

문제: 같은 "GDP"라도 명목/실질, 원계열/계절조정, 억원/백만원 등 단위가 다른 표가 섞여 있습니다. 기사에서 "GDP 성장률"이라고만 하면 이게 전기대비인지 전년동기대비인지 불명확한데, 표는 그 구분이 되어 있어야 합니다.

방향: 단위/기준 필드를 카탈로그에 강제하고, 매칭 후 검증 로직에서 "단위 호환성 체크"를 넣는 게 좋음.

작업 3 — 지역 계층 문제

문제: "서울 강남구 실업률" 같은 질의에서 표는 시도 단위까지만 있고 구 단위 데이터가 없는 경우가 있습니다. 지역명이 표에 없다고 그냥 카탈로그 밖으로 버려지면 억울한 케이스가 생길 수 있습니다(상위 지역 표로는 검증 가능한 경우도 있음).

방향: 지역 계층(시도-시군구-읍면동)을 코드 체계로 정규화해서 "요청 지역이 표의 최소 단위보다 세밀하면 조회불가, 아니면 집계 가능" 식으로 판단.

작업 4 — 비율 / 파생지표 문제

문제: 기사에 "출산율이 들었다"처럼 나오는데, 실제로 이건 KOSIS 단일 표가 아니라 출생아수/가임여성인구 두 표를 조합해야 나오는 파생지표인 경우가 많습니다. 벡터 검색은 단일 표 하나만 리턴하니 애초에 검증 불가능한 걸 억지로 매핑하려는 상황이 생길 수 있습니다.

방향: "직접 표 있음 / 파생계산 필요 / 검증불가" 3단계 분류 로직이 필요.

작업 5 — 분류체계 개편 (표는 안 죽었는데 컬럼/카테고리가 바뀜)

문제: 시점 문제랑 연결되는데, 표 자체는 계속 살아있어도 산업분류(KSIC), 직업분류(KSCO) 개편으로 특정 연도 전후 컬럼 구조가 달라지는 경우가 있습니다. 이건 "표가 없다"가 아니라 "표는 있는데 조회 로직이 달라져야 한다"는 문제라 더 까다롭습니다.

방향: (추가 조사 필요 — 조사연혁 API 등으로 개편 시점을 표별로 확인하는 방법 검토)

작업 6 — Silent failure / 낮은 확신도 매핑

문제: 가장 위험한 케이스입니다. 유사도가 애매하게 낮은데도(0.6~0.7대) top-1을 그냥 리턴해버리면, 사실검증 시스템이 틀린 근거로 "사실"이라고 판정할 수 있습니다. 뉴스 팩트체크 도메인 특성상 이게 제일 치명적입니다.

방향: threshold 밑이면 무조건 "검증불가/근거부족" 반환. (참고: agent/kosis/query_vdb.py의 VDB_MIN_SIMILARITY=0.75로 이미 부분 반영되어 있음 — 이 임계값이 적절한지 재검토 필요.)

작업 7 — 기사 표현과 통계 용어 간 괴리

문제: 기사는 "물가가 많이 뛰었다"처럼 구어체/함축적 표현을 쓰는데, 표는 "소비자물가지수(CPI)"라는 학술 용어로 되어 있습니다. 임베딩 모델이 도메인 특화가 안 되어 있으면 이 갭이 큼니다.

방향: 쿼리 확장(query expansion)이나 fine-tuning, 혹은 LLM으로 기사 표현을 통계 용어로 먼저 정규화하는 전처리 단계 추천.

5. 참고 파일 위치

파일	역할
agent/kosis/crawl_table_catalog.py	KOSIS 전체 표 크롤러 (완료, 재실행 불필요)
notebooks/vdb_embedding_colab.ipynb	표 이름 임베딩 생성 (완료)
agent/kosis/build_vdb_index.py	Chroma 적재 스크립트 (재구축 시 M/construction_ef 파라미터 조정)
agent/kosis/query_vdb.py	VDB 조회 모듈 (claim → 후보 표 리스트)
agent/mapping/reranker.py	리랭킹 로직 (_merge_candidates)
agent/pipeline/export_for_rerank.py	배치 파이프라인 1~3단계 (로컬)
agent/pipeline/resume_after_rerank.py	배치 파이프라인 4~8단계 (코랩 리랭킹 이후)
notebooks/reranker_colab.ipynb	리랭커(bge-reranker-v2-m3) 코랩 실행 노트북